Asynchronous Logger - Background Thread Logging

Theory, Edge Cases & Code Examples

- ✓ Logging Requirements
- ✓ Design: Background Thread + Queue
- ✓ Destruction with Pending Messages
- ✓ Disk Full / File Write Error
- ✓ Multi-Threaded Logging

→ *Swipe to learn* →

Asynchronous Logger - Background Thread Logging

Theory

📌 1. Logging Requirements

Features:

- Thread-safe (multiple threads log concurrently)
- Asynchronous (don't block caller)
- Levels (DEBUG, INFO, WARN, ERROR)
- Timestamping
- Formatting

Asynchronous Logger - Background Thread Logging

Theory

📌 2. Design: Background Thread + Queue (1/3)

```
Thread 1 ──┐
Thread 2 ──┼──> [Queue] ──> [Logger Thread] ──> File
Thread 3 ──┘
```

Producer threads: Push log messages to queue

Consumer thread: Pops messages, writes to file

```
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <fstream>
#include <sstream>
#include <chrono>
#include <iomanip>

enum class LogLevel {
    DEBUG, INFO, WARN, ERROR
};

class Logger {
private:
    struct LogMessage {
        LogLevel level;
        std::string message;
        std::chrono::system_clock::time_point timestamp;
    };

    std::queue<LogMessage> queue_;
    std::mutex mutex_;
    std::condition_variable cv_;
    std::thread worker_;
    bool stop_{false};

    std::ofstream file_;
    LogLevel min_level_{LogLevel::DEBUG};

    void worker_loop() {
// ...continues on next slide
```

Asynchronous Logger - Background Thread Logging

Theory

📌 2. Design: Background Thread + Queue (2/3)

```
// ...continued from previous slide
        while (true) {
            LogMessage msg;

            {
                std::unique_lock<std::mutex> lock(mutex_);
                cv_.wait(lock, [this]() {
                    return !queue_.empty() || stop_;
                });

                if (stop_ && queue_.empty()) {
                    return;
                }

                msg = std::move(queue_.front());
                queue_.pop();
            }

            write_to_file(msg);
        }
    }

    void write_to_file(const LogMessage& msg) {
        if (msg.level < min_level_) {
            return;
        }

        auto time_t = std::chrono::system_clock::to_time_t(msg.timestamp);
        auto tm = *std::localtime(&time_t);

        file_ << std::put_time(&tm, "%Y-%m-%d %H:%M:%S") << " ";
        file_ << "[" << level_to_string(msg.level) << "] ";
        file_ << msg.message << '\n';
        file_.flush();
    }

    static const char* level_to_string(LogLevel level) {
        switch (level) {
            case LogLevel::DEBUG: return "DEBUG";
            case LogLevel::INFO:  return "INFO";
            case LogLevel::WARN:  return "WARN";
// ...continues on next slide
```

Asynchronous Logger - Background Thread Logging

Theory

📌 2. Design: Background Thread + Queue (3/3)

[Code continues across slides]

```
// ...continued from previous slide
            case LogLevel::ERROR: return "ERROR";
            default: return "UNKNOWN";
        }
    }

public:
    explicit Logger(const std::string& filename, LogLevel min_level = LogLevel::DEBUG)
        : file_(filename, std::ios::app), min_level_(min_level)
    {
        worker_ = std::thread(&Logger::worker_loop, this);
    }

    ~Logger() {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            stop_ = true;
        }
        cv_.notify_one();
        worker_.join();
    }

    void log(LogLevel level, const std::string& message) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            queue_.push({level, message, std::chrono::system_clock::now()});
        }
        cv_.notify_one();
    }

    void debug(const std::string& msg) { log(LogLevel::DEBUG, msg); }
    void info(const std::string& msg)  { log(LogLevel::INFO, msg); }
    void warn(const std::string& msg)  { log(LogLevel::WARN, msg); }
    void error(const std::string& msg) { log(LogLevel::ERROR, msg); }
};
```

Asynchronous Logger - Background Thread Logging

Edge Cases

📌 1. Destruction with Pending Messages

Problem: Logger destroyed before queue empty.

Solution: Flush queue in destructor (our implementation does this).

Asynchronous Logger - Background Thread Logging

Edge Cases

📌 2. Disk Full / File Write Error

Problem: `file_.write()` fails.

Mitigation:

```
if (!file_) {
    std::cerr << "LOG ERROR: Failed to write\n";
}
```

Asynchronous Logger - Background Thread Logging

Code Examples

📌 1. Multi-Threaded Logging (1/2)

This example demonstrates **a production-ready asynchronous logging system handling concurrent log requests from multiple threads**. The code spawns 4 worker threads that each generate 10 log messages with 10ms delays, simulating a real application where multiple components log simultaneously without blocking each other or the main thread.

What this code does:
- Creates a Logger instance writing to "app.log" with minimum level INFO (DEBUG messages filtered)
- Logs application lifecycle events (startup, shutdown) from main thread
- Spawns 4 worker threads that each produce 10 log messages with unique worker IDs
- Each worker sleeps 10ms between logs to simulate real work being done
- All log messages are queued asynchronously and written by the background logger thread
- Destructor ensures all queued messages are flushed to disk before program exits

Key concepts demonstrated:
- **Thread-safe queueing** - multiple threads can call `logger.info()` simultaneously without data races
- **Asynchronous I/O** - worker threads return immediately from `log()` calls (~1μs), not blocked by disk I/O (~1ms)
- **Producer-consumer pattern** - worker threads produce log messages, background thread consumes and writes
- **RAII guarantee** - destructor flushes pending messages even if main() exits early
- **Log level filtering** - minimum level set to INFO means DEBUG calls would be silently ignored

Why this matters:
In high-throughput servers (web servers, databases, game servers), **synchronous logging is a performance killer**. If 1000 requests/second each log 5 messages, and each disk write takes 1ms, that's 5 seconds of blocked time per second - impossible! Asynchronous logging decouples application threads from I/O latency, allowing threads to continue working while logs are written in the background.

Performance implications:
- Worker threads spend only ~1μs in `log()` vs ~1ms for synchronous disk write (1000× faster)
- Background thread batches writes and can optimize disk I/O patterns
- Queue allows burst handling - temporary high log volume doesn't block application
- Small memory cost: log queue grows if production rate exceeds write rate

Real-world applications:
- **Web servers**: Request/response logging without slowing request handling
- **Game servers**: Player action logging at 60+ FPS without frame drops
- **Database systems**: Query logging without impacting transaction throughput
- **Distributed systems**: Trace logging for debugging without performance penalties

app.log:

Asynchronous Logger - Background Thread Logging

Code Examples

📌 1. Multi-Threaded Logging (2/2)

```
#include <iostream>

void worker(Logger& logger, int id) {
    for (int i = 0; i < 10; ++i) {
        logger.info("Worker " + std::to_string(id) + " iteration " + std::to_string(i));
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

int main() {
    Logger logger("app.log", LogLevel::INFO);

    logger.info("Application started");

    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back(worker, std::ref(logger), i);
    }

    for (auto& t : threads) {
        t.join();
    }

    logger.info("Application exiting");

    return 0;
}
```

```
['2026-03-24 12:00:00 [INFO] Application started\n2026-03-24 12:00:00 [INFO] Worker 0 iteration 0\n2026-03-24 12:00:00
```

What You Learned

Key Takeaways

- ✓ Logging Requirements
- ✓ Design: Background Thread + Queue
- ✓ Destruction with Pending Messages
- ✓ Disk Full / File Write Error

→ Swipe to learn →